# UNIVERSIDAD POLITÉCNICA DE ATLAUTLA

**INGENIERÍA EN TECNOLOGÍAS DE LA INFORMACIÓN E INNOVACIÓN DIGITAL**

# FUNDAMENTOS DE INTELIGENCIA ARTIFICIAL

## ACTIVIDAD 4. ALGORITMOS DE BÚSQUEDA CIEGA: BFS Y DFS

**Alumno:** Eduardo Parrilla Toledano

**Grupo:** 172

**Docente:** Mtro. Javier García Lara

1. Introducción

Los algoritmos de búsqueda ciega exploran un espacio de estados sin utilizar una heurística que indique qué tan cerca está cada estado de la meta. BFS explora primero los nodos más cercanos al inicio; DFS explora una rama hasta donde puede y después retrocede. Ambos se pueden aplicar a laberintos y grafos.

2. Planteamiento del problema

Se representa un laberinto mediante un grafo. La entrada es el nodo A y la salida es el nodo G. El objetivo es encontrar una ruta y comparar el orden de exploración de BFS y DFS. Para que el resultado sea reproducible, los vecinos se guardan en un orden fijo.

Grafo de ejemplo: A conecta con B y C; B conecta con D y E; C conecta con F; E y F conectan con G.

3. Diferencias entre BFS y DFS

| Criterio | BFS | DFS | Observación |
|---|---|---|---|
| Estructura | Cola FIFO | Pila LIFO o recursión | Define el orden de exploración |
| Exploración | Por niveles | Profundiza por ramas | Ambos visitan nodos alcanzables |
| Camino más corto | Sí, si las aristas tienen igual costo | No necesariamente | BFS encuentra el menor número de aristas |
| Memoria | Puede ser alta | Suele usar menos en grafos profundos | Depende del grafo |
| Complejidad | O(V+E) | O(V+E) | Con lista de adyacencia |

4. Código Python

```
from collections import deque

grafo = {
    "A": ["B", "C"],
    "B": ["D", "E"],
    "C": ["F"],
    "D": [],
    "E": ["G"],
    "F": ["G"],
    "G": []
}

def reconstruir_camino(padres, inicio, objetivo):
    if objetivo not in padres and objetivo != inicio:
        return None
    camino = []
    actual = objetivo
    while actual is not None:
        camino.append(actual)
        actual = padres[actual]
    return list(reversed(camino))
```

```
def bfs(grafo, inicio, objetivo):
    cola = deque([inicio])
    visitados = {inicio}
    padres = {inicio: None}
    orden = []

    while cola:
        nodo = cola.popleft()
        orden.append(nodo)

        if nodo == objetivo:
            break

        for vecino in grafo.get(nodo, []):
            if vecino not in visitados:
                visitados.add(vecino)
                padres[vecino] = nodo
                cola.append(vecino)

    return orden, reconstruir_camino(padres, inicio, objetivo)

def dfs(grafo, inicio, objetivo):
    pila = [inicio]
    visitados = set()
    padres = {inicio: None}
    orden = []

    while pila:
        nodo = pila.pop()
        if nodo in visitados:
            continue

        visitados.add(nodo)
        orden.append(nodo)

        if nodo == objetivo:
            break

        # Se invierte para respetar el orden definido en el grafo
        for vecino in reversed(grafo.get(nodo, [])):
            if vecino not in visitados:
                if vecino not in padres:
                    padres[vecino] = nodo
                pila.append(vecino)

    return orden, reconstruir_camino(padres, inicio, objetivo)

for nombre, algoritmo in [("BFS", bfs), ("DFS", dfs)]:
    orden, camino = algoritmo(grafo, "A", "G")
    print(nombre)
    print("Orden de exploración:", orden)
    print("Camino encontrado:", camino)
```

5. Resultados esperados

Con el grafo y el orden de vecinos definidos en el código, BFS explora por niveles y encuentra el camino A → B → E → G. DFS profundiza por la primera rama según el orden definido y encuentra una ruta válida; el orden exacto puede cambiar si se modifica la lista de vecinos.

```
BFS
Orden de exploración: ['A', 'B', 'C', 'D', 'E', 'F', 'G']
```

```
Camino encontrado: ['A', 'B', 'E', 'G']

DFS
Orden de exploración: ['A', 'B', 'D', 'E', 'G']
Camino encontrado: ['A', 'B', 'E', 'G']
```

6. Análisis de resultados

BFS utiliza una cola FIFO, por lo que visita primero los nodos que están a menor profundidad. Si todas las aristas tienen el mismo costo, encuentra un camino con el menor número de aristas. DFS utiliza una pila y profundiza en una rama antes de retroceder; puede encontrar una solución rápidamente, pero no garantiza el camino más corto.

7. Complejidad

Con una lista de adyacencia, BFS y DFS tienen complejidad temporal O(V + E), donde V es el número de vértices y E el número de aristas. Su memoria puede llegar a O(V), además de la representación del grafo. En grafos grandes, BFS puede almacenar muchos nodos en la cola.

8. Conclusión

BFS y DFS son métodos fundamentales para explorar espacios de búsqueda. BFS resulta adecuado cuando se necesita el camino con menor número de pasos en un grafo no ponderado. DFS es útil para recorrer estructuras y explorar ramas, aunque no garantiza una ruta mínima. La selección depende del objetivo y de las características del problema.

9. Referencias


- Russell, S., & Norvig, P. (2021). Artificial intelligence: A modern approach (4th ed.). Pearson.
- Python Software Foundation. (s. f.). collections — Container datatypes. Python documentation. https://docs.python.org/3/library/collections.html
- NetworkX. (s. f.). Traversal. https://networkx.org/documentation/stable/reference/algorithms/traversal.html